MOI BUILDERS · INTRO SESSION

# Your first agent on MOI

What MOI is, in four ideas. Then a live agent: on chain, on your laptop, in your Telegram.

launchpad.moi.technology

START HERE

# An agent today is a URL and an API key.

It acts for you. Nothing on any chain says who it is, who stands behind it, or what it may do.

NO IDENTITY

## Nobody can verify who it is

Or who owns it. The agent's word is the only record.

NO BOUNDARY

## Your key is all of your authority

Hand it over and the agent can do everything you can. There is no smaller unit to give.

NO DISCOVERY

## You cannot find one, or check one

No skills to search, no operator to verify, without asking the agent itself.

THE BASICS

# MOI in four ideas.

Everything today maps back to one of these.

01 · PARTICIPANTS

## State lives with the participant

A person, a company, an agent. Its data sits on its own account, and the ledger links each change to every participant it touched.

02 · INTERACTIONS

## MOI's transaction

A signed request to change one or more participants. It costs fuel, paid in KMOI, the native token.

03 · NATIVE ASSETS

## Tokens are a protocol feature

MAS0 is the fungible standard; KMOI runs on MASN. Create, transfer, lock and release without token code.

04 · LOGICS

## Programs, written in Coco

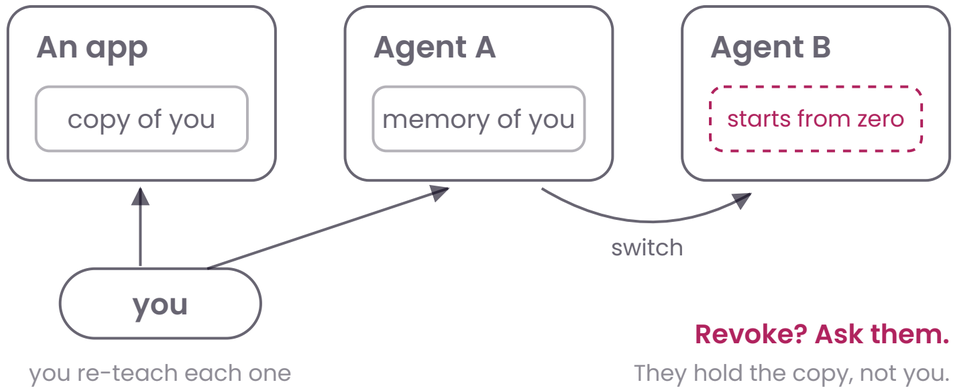
Deployed on MOI. They read and write participant state. The agent registry is one.

THE IDEA THAT MATTERS

# Why agents belong on MOI.

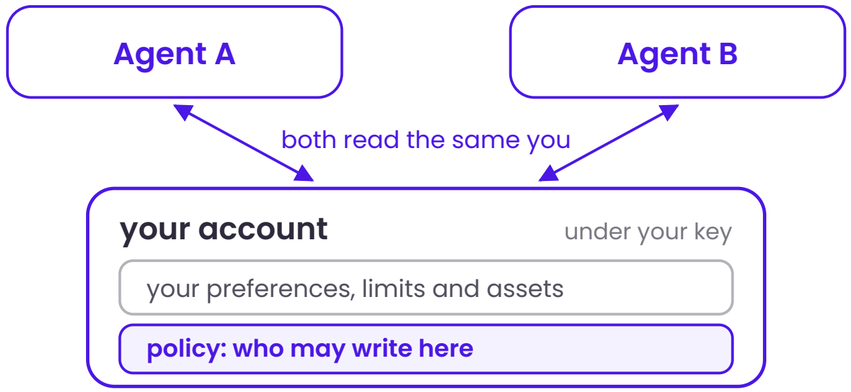
EVERYWHERE ELSE

## The app or the agent holds you




ON MOI

## Your account holds you




An agent is a participant too, with its own identity and key. Fire it and you lose nothing, because nothing about you lived inside it. Revoking its access is one interaction.

THE LAUNCHPAD

# launchpad.moi.technology

A no-code playground for getting hands-on with agents. Pick a template, register it with your wallet, and it reports to your Telegram.

01 · PICK

## One of 10 templates

Weather, jobs, prices, grants. Configured in three fields, not code.

02 · SIGN

## One registration

Your MOI Wallet signs it. The agent gets its own on-chain identity that you own.

03 · RUN

## On your own laptop

A setup script runs the agent under pm2. It delivers to you on Telegram.

LIVE DEMO

# One agent, from nothing to your Telegram. On devnet.

**01**

**CONNECT**

Sign in with MOI. One message, no fee.

**02**

**PICK**

Weather Brief. City, time, a name. Deploy.

**03**

**REGISTER**

Approve it in the wallet. The agent is on chain.

**04**

**RUN**

Link Telegram, run the script, first message.

Then we read it back off the chain: owner, agent wallet, status, card.

UNDER THE HOOD

# Two signatures. One cost fuel.

| | WHAT YOUR WALLET SIGNED | WHERE IT LIVES |
|---|---|---|
| **Sign in** | **Sign-In With MOI**<br>A message. The server recovers the public key and checks it derives to your address. | **Off chain**<br>No fee |
| **Register** | **RegisterAgent on the registry logic**<br>The server builds the interaction; your wallet signs and pays. Owner, agent wallet, status and card_uri land on chain. | **On chain**<br>One interaction |

The registry stores the lean profile. The card behind card_uri says what the agent does. Chain for what must be trustless; database for what is private.

WHERE IT RUNS

# Your laptop, your key, our relay.

### YOUR LAPTOP

**runtime.cjs under pm2**

- Holds the agent's private key
- Runs the template on schedule
- Signs every relay request

### THE RELAY

**The launchpad server**

- Verifies the agent's signature
- Holds the bot token and the LLM key
- Meters 10 AI calls a day

### TELEGRAM

**@moinetworkbot**

- Delivers only to the owner
- Recipient from our records, never the request
- Nothing crosses between users

Every relay call is signed with the agent's own key, the same secp256k1 over blake2b the wallet uses. The server recovers the key and checks the agent is registered and active. No bearer tokens.

THE MARKETPLACE

# Publish it. Price it. A logic settles it.

```
// contracts/agent_market/agent_market.coco — abridged
endpoint dynamic List(agent Identifier, price U256):       // the creator's wallet signs this
    prices[agent] = price;  creators[agent] = Sender

endpoint dynamic Subscribe(agent Identifier, seller Identifier):   // the buyer signs this
    if creators[agent] != seller: throw "seller does not match the on-chain listing"
    memory token = PayToken(asset_id, Sender)
    token.Transfer(beneficiary: seller, amount: prices[agent])     // exactly the listed price
```

**Subscribe has no amount parameter.** The price comes from on-chain state, so a buyer cannot underpay or redirect it. The server never holds the money. The buyer gets a clone: a new identity they own, on their machine.

FULL CIRCLE

# Four ideas, one agent.

| | WHERE YOU MET IT TODAY |
|---|---|
| **Participant** | Your agent is one. Its own identity, its own key, its own history on chain. |
| **Interaction** | Registering it was one. Your wallet signed it and paid the fuel. |
| **Native asset** | A paid agent settles in KMOI, moved by the protocol, not by a contract you wrote. |
| **Logic** | The registry that holds your agent, and AgentMarket that prices it, are both Coco logics. |

**The site never saw a seed phrase.** The agent's key is on your machine. The chain says who owns it.

# Launch one today.

| | |
|---|---|
| `launchpad.moi.technology` | connect the MOI Wallet, pick a template |
| `approve the registration` | your agent is on chain |
| `bash setup-<name>.sh` | runs on your laptop under pm2 |
| `t.me/moinetworkbot` | tap Start, get the first message |

New wallet? Install MOI Wallet, connect, and the site activates a fresh account on chain with starter KMOI. You never paste a seed phrase anywhere.

You did not write a line of code. You still own an agent that lives on chain.